Design and Implementation of a Browser-Based Real-Time Object Detection System Using TensorFlow.js & COCO-SSD Model

Student name: Shiv Trivedi

Student ID: 100536444

Contents

1. Introduction 3

2.Aims & Objectives 3

2.1 Aims 3

2.2 Objectives 3

3. Background 4

3.1 Object Detection 4

3.2 COCO-SSD Model 4

3.3 TensorFlow.js 5

4. System Architecture 5

4.1 Server-Side Architecture 5

4.2 Client-Side Architecture 5

5. Implementation 6

5.1 Load the Model 6

5.2 Handling of Media Sources 6

5.3 Object Detection Workflow 6

5.4 Visualisation and User Interface Design 7

6. Performance Analysis and Evaluation 7

7. Interpretation of the Accuracy and Confidence Score 7

8. Error Management, System Constructs 8

9. Professional and Academic Significance 8

10. The Future Development Opportunities 8

11. Conclusion 9

12. Reference list 10

1. Introduction

The development of Artificial Intelligence (AI) and browser-based computing has had a major impact in the process of developing and deploying intelligent applications. The development of libraries like TensorFlow.js has meant that now more complex deep learning models can be run in web browsers; they have no longer relied on the more powerful server or dedicated hardware of GPUs (Huang et al., 2016). The project is based on the creation of a real-time object detection system deployed as a web application built on the TensorFlow.js library and the COCO-SSD pre-trained model with the assistance of the lightweight Node.js and Express server to provide the application. The system allows the user to either use a live webcam feed to detect the objects or one can upload a static image to achieve the same purpose. Detection objects are identified with visual highlights represented by bounding boxes with class labels and confidence scores, and hence the results are easily interpreted by the user (Leng and Liu, 2020). This makes the application more scalable, less latent, and more user friendly as all the inference operations are carried out on the client side such that the visual data is not sent to the server. In addition, the project shows how the latest web technologies, which include HTML5 media APIs, machine learning with JavaScript, and interactive user interfaces, can be integrated to create effective and interactive computer vision systems. Live-time performance measurements, such as Frames Per Second (FPS), are also added to consider the system in a live situation to determine its responsiveness and usability (Anbarasi, 2025).

2.Aims & Objectives

2.1 Aims

This project is designed to create a real-time and web-based application of the object detector based on the TensorFlow.js. It tries to illustrate how the pre-trained machine learning models can be integrated with modern web technologies to provide an efficient, interactive, and privacy-conscious approach to computer vision (Li and Liu, 2019).

2.2 Objectives

- To design and develop a real time object detection system that can be fully used in a web browser platform.

- To prepare and integrate the COCO-SSD pre-trained deep learning model to execute object recognition tasks with the help of TensorFlow.js.
- To be able to use modern browser API to process real-time webcam input and upload of static images.
- To show the object detection results in a visual form that is easy to understand and interpret, in the form of bounding boxes, label, and confidence levels.
- To check the indicators of performance including FPS to evaluate the efficiency of the real-time execution.
- To implement professional software development methods, such as modular codifying and documentation and reliable error handling.
- To promote the essential learning outcomes in the area of Frontend Development, Web-based Artificial Intelligence and professional computing practices.

3. Background

3.1 Object Detection

Detecting objects is a basic field of computer vision which is concerned with detecting several objects in visual information and at the same time ascertaining the spatial positions of those objects. Object detection is used to identify a variety of object classes with bounding box coordinates in contrast to image classification, which typically gives only one type of object to an image as a whole. This is necessary in intelligent surveillance systems, autonomous driving, traffic analysis, and smart environment monitoring, among other applications, and recognition and localisation are needed (Li, Wang and Wang, 2022).

3.2 COCO-SSD Model

COCO-SSD is a popular pre-trained deep learning model that is used to detect objects efficiently. It is trained with the Common Objects in Context (COCO) dataset of hundreds of thousands of annotated images of a wide variety of common object categories. The Single Shot Detector (SSD) architecture is a single forward pass object localisation and classification architecture that is thus especially well adapted to real-time object localisation and classification in resource-constrained applications, such as web browsers(Li, Yang and Zhou, 2017).

3.3 TensorFlow.js

TensorFlow.js is a machine learning framework, written in open-source and implemented in JavaScript, which allows trained models to run in both web browsers and in a Node.js application. It enables several execution backends such as CPU and GPU acceleration with WebGL allowing models to scale to hardware resources available. The library gives freedom to the developers to either load already trained models, fine-tune those models, or create their own neural networks to be deployed into web-based applications (Liu et al., 2016).

4. System Architecture

4.1 Server-Side Architecture

The back-end of the application is written on Node.js and on the Express.js framework. Its main purpose is to serve and send passive frontend resources such as HTML, CSS, and JavaScript files to the web browser of the client. The server is used to offer a basic execution platform through which the application can be developed and tested locally using a browser. Notably, no inference or data processing of machine learning is carried out in the server. The entire process of object detection is performed on the client side thereby minimizing the infrastructure, minimizing the operational costs and enhancing scalability. This method leads to increased privacy of the user as well since the image and video data are not sent to the server and are stored in the user's device.

4.2 Client-Side Architecture

All user interaction, data processing and object detection functionality are handled by the frontend. The prepared COCO-SSD model is loaded and brought online in the browser with the help of TensorFlow.js. Contributions by media is provided with the help of the latest web APIs, including the Media Devices API, which streams live webcams and the standard file input mechanism, which uploads the static images. Every image frame in a video or uploaded image is fed through the object detection model to be inferred. The findings of the detection are displayed visually on an HTML5 canvas, with bounding boxes, name of the object class and confidence score being superimposed over the media content. Furthermore, real-time performance indicators such as Frames Per Second (FPS) and detected objects are also computed and shown to give an idea of the system responsiveness and execution efficiency (Managwu, C., Kushchu, I., & Matthias, D, 2024).

5. Implementation

5.1 Load the Model

The application includes the COCO-SSD object detection model, which is loaded asynchronously when the web page is loaded using TensorFlow.js. In the process a visual loading indicator is shown to indicate the user of the status of model initialisation. After loading, an initial pass with a dummy input to perform the inference takes place in order to initialise internal tensors and optimise run-time. This warm-up process plays a major role in minimizing the delay that is experienced during the initial real-time identification. In order to ensure effective memory utilization, tensors generated in the middle of the inference are explicitly dropped, avoiding memory growth over time when using the web camera.

5.2 Handling of Media Sources

The application allows live and static media inputs. The browser getUserMedia() interface allows live video capture and the use of the device camera needs the express approval of the user. The controls are also availed to enable users to start and stop the video stream when the need arises. Besides the live video, users can use a file selection interface where they can upload static images in popular formats, e.g., JPEG and PNG. Images uploaded will be automatically processed on the moment of their selection, allowing one-frame recognition of an object. This two-input solution proves to be effective in terms of the modern web API usage to work with various media sources.

5.3 Object Detection Workflow

Object detection is implemented in the form of a processed pipeline scheme applied to every frame of video or uploaded image. The media frame is initially obtained and coded into a format which can be understood by the detection model. After that, it is sent to the detect() method offered by the COCO-SSD model, which returns a list of predictions. The predicted objects are represented by each prediction with the category of the detected object, the confidence score, and the coordinates of the bounding box. In order to enhance the reliability of the results, predictions with a confidence level lower than a specified confidence level are eliminated. The rest of the detections are sent to be visual rendered (Managwu, 2024).

5.4 Visualisation and User Interface Design

Findings in detection are processed using HTML5 Canvas API, which is used to overlay graphical data on the video stream or image. A bounding box, and the name of the class and the percentage of the detected object are displayed with every identified object. The user interface also involves other interactive elements like a confidence threshold bar to modify the sensitivity of the detection system, a real-time frame rate display to indicate the performance of the system and finally a summary panel which displays the total object count detected. The interface will be responsive enough so that it can be used and accessed by desktop, tablet and mobile platforms.

6. Performance Analysis and Evaluation

The performance of the system is also measured mostly by use of Frames Per Second (FPS) which is an indication of the image frame rate at which the application processes the images to detect an object per second. Various factors affect FPS among them being the computing power of the device used by a user, video or image resolution chosen, and the number of contents in each frame. The application enables real-time FPS display and, therefore, gives real-time feedback on system responsiveness and performance efficiency. This performance awareness is a critical notion in web-based AI systems, since it allows the users and developers to realize the trade-offs among detection quality and real-time constraint of processing (Redmon and Farhadi, 2016).

7. Interpretation of the Accuracy and Confidence Score

When applying to this application it is worth noting the difference between the overall model accuracy and confidence scores which are reported during detection. Confidence scores are the estimated likelihood of the model that an object that has been detected is a member of a specific class in a given frame. These values show reliability of individual predictions but they are not used to determine the actual accuracy of the model. Formal accuracy evaluation is comparison to labelled ground-truth datasets and can be done offline, as part of training or benchmarking. This project is real-time and in-browser-based, and thus accuracy metrics are not calculated, and confidence scores serve as the main metric of detection reliability (Shen et al., 2017).

8. Error Management, System Constructs

The application has basic mechanisms of error management to enhance stability and user interaction. When there are some critical problems, user-friendly notifications are shown (the failure to load the object detection model, the camera permission denied and so on). In order to avoid execution errors, object detection is simply turned off until the model is fully initialised. Though these have been done, there are limitations of the system. Some of the external factors such as unfavourable lighting conditions, browser implementation variations, and hardware differences may influence detection performance. Also, the model can identify only the predefined object categories that are present in the COCO dataset, which limits its use in domain-specific situations.

9. Professional and Academic Significance

The project can be connected to several of the essential technical skills in the development of software and artificial intelligence (AI) that are needed today. It proves the usefulness of the combination of frontend web technologies and machine learning frameworks in providing an interactive and real-time application. The importance is put on long-term scalable and maintainable code practices, modular architecture, and straightforward documentation, which facilitate long-term scalability and cooperative development. Scholarly, the project offers practical knowledge on Web-based Artificial Intelligence, client-side computer vision, and model deployment approaches and is essentially a way of linking theory and practice.

10. The Future Development Opportunities

It is possible to introduce several improvements to expand the application. Introduction of object tracking between successive video time frame would allow the consistent recognition of identified objects with time. The efficiency might be enhanced with an addition of region-of-interest processing that enables inference to concentrate on the areas of interest in every frame. Other usability functions like saving pictures with annotated detections might also be added. Moreover, a more in-depth analysis of the accuracy and the speed trade-offs would be possible with the help of comparative performance analysis based on other detection models like the YOLO-based architecture throughout the support of the TensorFlow.js platform.

11. Conclusion

This project is able to show the processes of creating a real-time, browser-based object detector with TensorFlow.js and the pre-trained COCO-SSD model. The application is scalable, has low latency and enhances data privacy by doing all inference operations on the client side without the use of server-side computation. The solution makes a successful blend of the current web technologies with machine learning, which satisfies the requirements of the project and demonstrates the feasibility of Web AI in real-time computer vision applications.

12. Reference list

Anbarasi (2025). Object Detection using Tensorflow Js. *International Journal of Advanced Research in Education and TechnologY(IJARETY) IJARETY ©2025 | An ISO*, [online] 12(3), p.1049. doi:https://doi.org/10.15680/IJARETY.2025.1203044.

Huang, J., Rathod, V., Sun, C., Zhu, M., Korattikara, A., Fathi, A., Fischer, I., Wojna, Z., Song, Y., Guadarrama, S. and Murphy, K. (2016). *Speed/accuracy trade-offs for modern convolutional object detectors*. [online] arXiv.org. Available at: https://arxiv.org/abs/1611.10012? [Accessed 12 Jan. 2026].

Leng, J. and Liu, Y. (2020). Single-shot augmentation detector for object detection. *Neural Computing and Applications*. doi:https://doi.org/10.1007/s00521-020-05202-0.

Li, W. and Liu, G. (2019). *A Single-shot Object Detector with Feature Aggragation and Enhancement*. [online] arXiv.org. Available at: https://arxiv.org/abs/1902.02923? [Accessed 12 Jan. 2026].

Li, Y., Wang, L. and Wang, Z. (2022). Single-Shot Object Detection via Feature Enhancement and Channel Attention. *Sensors*, 22(18), pp.6857–6857. doi:https://doi.org/10.3390/s22186857.

Li, Z., Yang, L. and Zhou, F. (2017). *FSSD: Feature Fusion Single Shot Multibox Detector*. [online] arXiv.org. Available at: https://arxiv.org/abs/1712.00960? [Accessed 12 Jan. 2026].

Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y. and Berg, A.C. (2016). SSD: Single Shot MultiBox Detector. *Computer Vision – ECCV 2016*, [online] 9905(5), pp.21–37. doi:https://doi.org/10.1007/978-3-319-46448-0_2.

Managwu, C., Kushchu, I., & Matthias, D (2024). *Browser-Based Object Detection System for Isolating Plastic Bottles using the COCO-SSD Model. - British Journal of Computer, Networking and Information Technology (BJCNIT)*. [online] British Journal of Computer, Networking and Information Technology (BJCNIT). Available at: https://abjournals.org/bjcnit/papers/volume-7/issue-4/browser-based-object-detection-system-for-isolating-plastic-bottles-using-the-coco-ssd-model/? [Accessed 12 Jan. 2026].

Managwu, C. (2024). BROWSER-BASED OBJECT DETECTION SYSTEM FOR ISOLATING PLASTIC BOTTLES USING THE COCO-SSD MODEL. *international journal*

of advanced research in computer science, 15(5), pp.50–52. doi:https://doi.org/10.26483/ijarcs.v15i5.7124.

Redmon, J. and Farhadi, A. (2016). *YOLO9000: Better, Faster, Stronger*. [online] arXiv.org. Available at: https://arxiv.org/abs/1612.08242? [Accessed 12 Jan. 2026].

Shen, Z., Liu, Z., Li, J., Jiang, Y.-G., Chen, Y. and Xue, X. (2017). *DSOD: Learning Deeply Supervised Object Detectors from Scratch*. [online] arXiv.org. Available at: https://arxiv.org/abs/1708.01241? [Accessed 12 Jan. 2026].